

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 8

TITLE:

Design and develop a Smart Inventory Management System for Construction Sites. The system tracks materials and inventory movement to improve resource utilization and site management efficiency.

TEAM MEMBERS:

HARISANKARAN S
(24MER013)

MUTHUKUMAR M
(24MER034)

MONESHWARAN C
(24MER032)

PROJECT OVERVIEW

Construction sites handle large quantities of materials such as cement, steel, bricks, sand, pipes, electrical items, tools, fasteners and other construction components. Proper inventory management is important because material shortages, excess stock, misplaced materials and improper recording can affect project cost and completion time.

The proposed Smart Inventory Management System uses an ESP32 microcontroller, RFID technology, a load cell with HX711 module, Wi-Fi communication and a cloud monitoring platform. The system identifies construction materials, measures their quantity and monitors material movement between storage and usage areas.

The collected inventory data is transmitted to the cloud platform through Wi-Fi. The data can be stored and displayed using a dashboard for real-time monitoring and historical analysis. The system can also generate alerts when the available material quantity falls below a predefined minimum stock level.

PROBLEM STATEMENT

Construction sites require continuous movement of materials from storage areas to different working locations. In conventional inventory systems, material quantities and movements are commonly recorded manually.

Manual inventory management can result in:

- Incorrect material quantity records.
- Delay in updating stock information.
- Material shortages during construction activities.
- Excess or unused inventory.
- Difficulty in tracking material movement.
- Misplacement of construction materials.
- Increased dependence on manual inspection.

Therefore, a smart system is required to monitor construction materials, track inventory movement and provide real-time information about the available stock.

PROPOSED SOLUTION

The proposed Smart Inventory Management System uses an ESP32 controller as the main processing unit.

RFID tags are assigned to construction materials or material storage containers. An RFID reader identifies the material when it enters or leaves the inventory area.

A load cell connected to an HX711 module is used to measure the quantity or weight of the material. The ESP32 processes the RFID identification and load-cell measurement.

The inventory information is then transmitted through Wi-Fi to a cloud platform such as Adafruit IO. The cloud dashboard displays the current material quantity, movement status and stock condition.

When the quantity of a material reaches the minimum stock level, the system generates a LOW STOCK alert. If the material quantity reaches zero, the system identifies the condition as OUT OF STOCK.

WORKING PRINCIPLE

The system works by combining material identification and quantity measurement.

The RFID reader identifies the material or storage container using its RFID tag. The load cell measures the quantity or weight of the material. The ESP32 receives the sensor information and processes the data.

The processed data follows the sequence:

RFID TAG → RFID READER → ESP32 → LOAD CELL + HX711 → Wi-Fi → CLOUD PLATFORM → INVENTORY DASHBOARD

The ESP32 compares the measured quantity with the predefined minimum stock value.

The inventory condition is classified as:

NORMAL – Sufficient material is available.

LOW STOCK – Material quantity has reached the minimum stock level.

OUT OF STOCK – Material quantity has reached zero or the critical reserve level.

The cloud dashboard provides remote access to the inventory information.

SYSTEM ARCHITECTURE

Construction Material / RFID Tag

↓

RFID Reader

↓

ESP32 Microcontroller

↓

Load Cell + HX711 Module

↓

Wi-Fi Network

↓

Cloud Platform

↓

Inventory Database / Cloud Feeds

↓

Web / Mobile Dashboard

The ESP32 acts as the central controller. The RFID reader provides material identification, while the load cell provides quantity measurement.

The Wi-Fi network transfers the processed inventory data to the cloud platform. The cloud dashboard provides real-time visualization and historical inventory analysis.

INVENTORY THRESHOLD VALUES

Inventory Condition Description

NORMAL	Quantity is above the minimum stock level
LOW STOCK	Quantity reaches or falls below the minimum stock level
CRITICAL	Quantity reaches the critical reserve level
OUT OF STOCK	Quantity reaches zero

The minimum stock value can be configured according to the material type.

For example:

Cement – Minimum stock: 10 kg

Steel – Minimum stock: 20 kg

Sand – Minimum stock: 25 kg

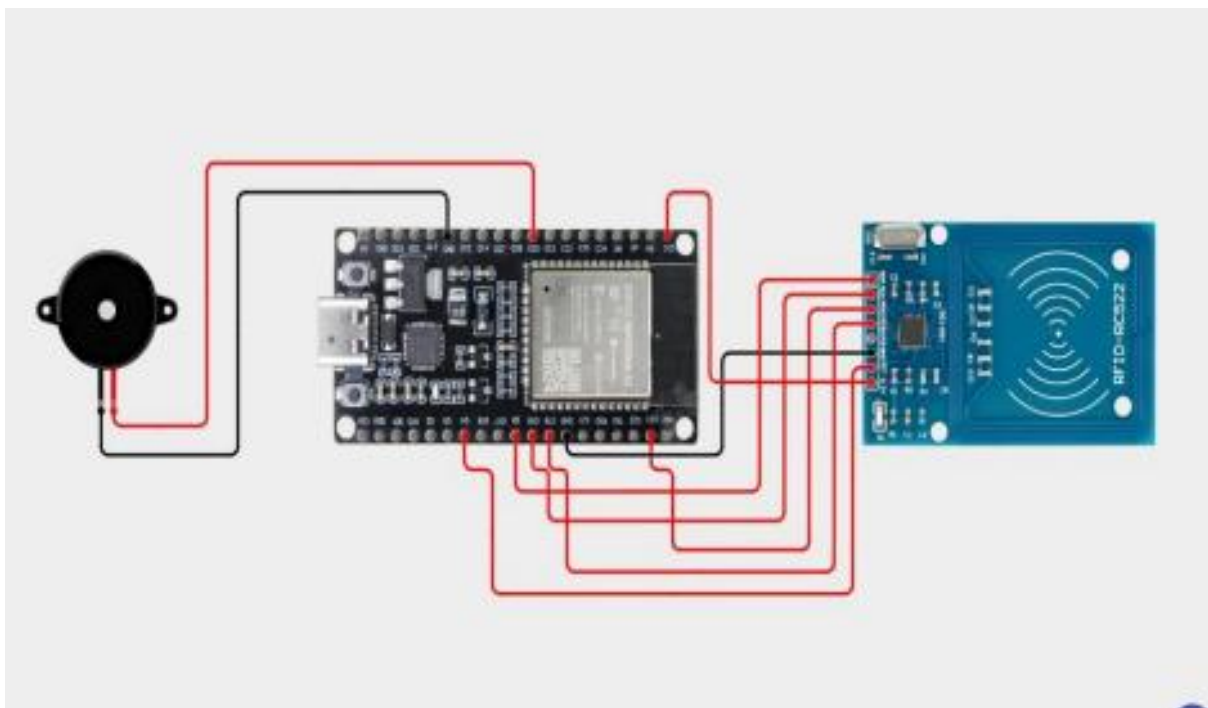
Bricks – Minimum stock: 50 units

These values are only prototype examples and can be changed according to actual construction requirements.

PARAMETERS MONITORED

Parameter	Sensor / Device	Purpose
Material ID	RFID Reader	Identifies the material
Material Name	ESP32 / Cloud	Displays the material name
Material Quantity	Load Cell + HX711	Measures material quantity
Movement Type	ESP32	Identifies stock-in or stock-out
Stock Status	ESP32 / Cloud	Identifies inventory condition
Storage Location	RFID / Cloud	Identifies storage area
Movement Time	Cloud	Records inventory movement time
IoT Connection	ESP32 Wi-Fi	Monitors cloud connection
Alert Status	Buzzer / Cloud	Indicates low stock condition

CIRCUIT DESIGN



ALGORITHM

- START
- Initialize the ESP32 microcontroller.
- Initialize the RFID reader.
- Initialize the load cell and HX711 module.
- Initialize the OLED display.
- Initialize the buzzer or LED indicator.
- Connect the ESP32 to the Wi-Fi network.
- Connect to the cloud platform.
- Scan for an RFID tag.
- Identify the material associated with the RFID tag.
- Read the load-cell measurement.
- Calculate the material quantity.
- Compare the measured quantity with the minimum stock value.
- Determine the inventory condition as NORMAL, LOW STOCK or OUT OF STOCK.
- Detect the material movement.
- Determine whether the movement is STOCK-IN, STOCK-OUT or TRANSFER.
- Display the material information on the OLED.
- Send the inventory information to the cloud.
- Activate the alert when the stock level becomes critical.
- Repeat the process continuously.
- STOP

CODING

```
//
=====

= // SMART INVENTORY MANAGEMENT SYSTEM

// ESP32 + MFRC522 RFID + BUZZER + ADAFRUIT IO //
=====

=

#include <WiFi.h>

#include <SPI.h>

#include <MFRC522.h>

#include "AdafruitIO_WiFi.h"

//
=====

= // WIFI DETAILS

//
=====

=

#define WIFI_SSID "OnePlus Nord CE5 7C5C"

#define WIFI_PASS "qkts5737"

//
=====

= // ADAFRUIT IO DETAILS

//
=====

=

#define IO_USERNAME "Hair_13"

#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

//
=====

= // ADAFRUIT IO CONNECTION
```

```

//
=====

= AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,
WIFI_PASS);

//
=====

= // ADAFRUIT IO FEEDS

//
=====

=

AdafruitIO_Feed *rfidUID =
io.feed("rfid-uid");

AdafruitIO_Feed *materialName =
io.feed("material-name");

AdafruitIO_Feed *inventoryStatus =
io.feed("inventory-status");

AdafruitIO_Feed *rfidDetection =
io.feed("rfid-detection");

AdafruitIO_Feed *scanCount =
io.feed("scan-count");

//
=====

= // ESP32 RFID PINS

//
=====

= //

// MFRC522 ESP32

// -----

// SDA/SS -> GPIO 5

```



```

// SCK -> GPIO 18
// MOSI -> GPIO 23
// MISO -> GPIO 19
// RST -> GPIO 22
// 3.3V -> 3.3V
// GND -> GND

//

//
=====
=

#define SS_PIN 5
#define RST_PIN 22
#define BUZZER_PIN 4
MFRC522 rfid(SS_PIN, RST_PIN);

//
=====
= // INVENTORY VARIABLES

//
=====
= int totalScans = 0;

// true = material is IN
// false = material is OUT
bool cementStatus = true;
bool steelStatus = true;
bool bricksStatus = true;

//
=====
= // FUNCTION DECLARATION

```

```
//  
=====
```

=

```
// IMPORTANT: This fixes the  
// "beepBuzzer was not declared in this scope" error.  
void beepBuzzer(int times);  
  
//  
=====
```

= // SETUP

```
//  
=====
```

=

```
void setup()  
{  
  Serial.begin(9600);  
  delay(1000);  
  
  // =====  
  // BUZZER  
  
  //  
  =====  
  
  pinMode(BUZZER_PIN, OUTPUT);  
  digitalWrite(BUZZER_PIN, LOW);  
  
  // =====  
  // START SPI  
  
  // =====  
  SPI.begin(18, 19, 23, 5);  
  
  // =====  
  // START RFID  
  
  // =====  
  rfid.PCD_Init();
```

```

delay(500);

Serial.println();

Serial.println("=====");
Serial.println(" SMART INVENTORY MANAGEMENT SYSTEM");
Serial.println(" ESP32 + MFRC522 + ADAFRUIT IO");
Serial.println("=====");

// =====
// RFID TEST
// =====

byte version = rfid.PCD_ReadRegister(rfid.VersionReg);

Serial.print("MFRC522 Version: 0x");

Serial.println(version, HEX);

if (version == 0x00 || version == 0xFF)
{
    Serial.println("ERROR: MFRC522 NOT DETECTED!");
    Serial.println("Check RFID wiring.");
}
else
{
    Serial.println("MFRC522 detected successfully.");
}

// =====
// CONNECT TO ADAFRUIT IO
//
=====

Serial.println();

Serial.println("Connecting to Adafruit IO...");

io.connect();

```

```

while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}
Serial.println();
Serial.println("Adafruit IO Connected!");

// =====
// INITIAL CLOUD STATUS
// =====
rfidDetection->save("NO TAG");
inventoryStatus->save("READY");
scanCount->save(totalScans);
Serial.println();
Serial.println("RFID Reader Ready");
Serial.println("Scan RFID Tag...");
Serial.println();
}

//
=====

= // LOOP

//
=====

=

void loop()
{
  // =====
  // KEEP ADAFRUIT IO CONNECTION ALIVE

```

```

// =====
io.run();

// =====
// CHECK FOR NEW RFID CARD

// =====

if (!rfid.PICC_IsNewCardPresent())
{
return;
}

// =====
// READ RFID CARD

//
=====

if (!rfid.PICC_ReadCardSerial())
{
return;
}

Serial.println("-----");
Serial.println("RFID CARD DETECTED!");

// =====
// READ UID

// =====
String uidString = "";
for (byte i = 0; i < rfid.uid.size; i++)
{
if (i > 0)
{
uidString += " ";

```

```

    }
    if (rfid.uid.uidByte[i] < 0x10)
    {
        uidString += "0";
    }
    uidString += String(rfid.uid.uidByte[i], HEX);
}
uidString.toUpperCase();

// =====
// DISPLAY UID

//
=====

Serial.print("RFID UID: ");
Serial.println(uidString);

// =====
// SEND RFID DETECTION

// =====
rfidDetection->save("DETECTED");

// =====
// SEND UID TO ADAFRUIT IO

// =====
rfidUID->save(uidString);

// =====
// INCREASE SCAN COUNT

// =====
totalScans++;

scanCount->save(totalScans);

// =====
// MATERIAL IDENTIFICATION

```

```

//
=====

String material = "UNKNOWN";
String status = "UNKNOWN";

// =====
// CEMENT RFID

// =====
if (uidString == "23 4A 91 7C")
{
    material = "CEMENT";
    if (cementStatus == true)
    {
        status = "OUT";
        cementStatus = false;
    }
    else
    {
        status = "IN";
        cementStatus = true;
    }
}

// =====
// STEEL RFID

//
=====

else if (uidString == "11 22 33 44")
{
    material = "STEEL";

```

```
if (steelStatus == true)
{
status = "OUT";
steelStatus = false;
}
else
{
status = "IN";
steelStatus = true;
}
}

// =====
// BRICKS RFID
// =====

else if (uidString == "55 66 77 88")
{
material = "BRICKS";
if (bricksStatus == true)
{
status = "OUT";
bricksStatus = false;
}
else
{
status = "IN";
bricksStatus = true;
}
```



```
}  
  
// =====  
// UNKNOWN RFID  
  
//  
=====
```

else

```
{  
material = "UNKNOWN";  
status = "UNAUTHORIZED";  
}  
  
// =====  
// SERIAL OUTPUT  
  
//  
=====
```

Serial.print("Material: ");
Serial.println(material);
Serial.print("Inventory Status: ");
Serial.println(status);

```
// =====  
// SEND DATA TO ADAFRUIT IO  
  
// =====  
materialName->save(material);  
inventoryStatus->save(status);  
  
// =====  
// BUZZER  
  
//  
=====
```

if (material == "UNKNOWN")

```

{
// Two beeps for unauthorized RFID
beepBuzzer(2);
}
else
{
// One beep for valid RFID
beepBuzzer(1);
}
Serial.println("Data sent to Adafruit IO");
Serial.println("-----");
// =====
// STOP RFID COMMUNICATION
//
=====

rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();
delay(1000);
// =====
// RESET CLOUD DETECTION STATUS
//
=====

rfidDetection->save("NO TAG");
}
//
=====
= // BUZZER FUNCTION

```

```

//
=====

=

void beepBuzzer(int times)
{
  for (int i = 0; i < times; i++)
  {
    digitalWrite(BUZZER_PIN, HIGH);
    delay(200);
    digitalWrite(BUZZER_PIN, LOW);
    delay(200);
  }
}

RFID NUMBER //
=====

= // SMART INVENTORY MANAGEMENT SYSTEM

// ESP32 + MFRC522 RFID + BUZZER + ADAFRUIT IO

//
=====

=

#include <WiFi.h>
#include <SPI.h>
#include <MFRC522.h>
#include "AdafruitIO_WiFi.h"

//
=====

= // WIFI DETAILS

```

```
//
=====

=

#define WIFI_SSID "OnePlus Nord CE5 7C5C"
#define WIFI_PASS "qkts5737"

//
=====

= // ADAFRUIT IO DETAILS

//
=====

=

#define IO_USERNAME "Hair_13"
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

//
=====

= // ADAFRUIT IO CONNECTION

//
=====

= AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,
WIFI_PASS);

//
=====

= // ADAFRUIT IO FEEDS

//
=====

=

AdafruitIO_Feed *rfidUID =
io.feed("rfid-uid");

AdafruitIO_Feed *materialName =
io.feed("material-name");

AdafruitIO_Feed *inventoryStatus =
```

```

io.feed("inventory-status");
AdafruitIO_Feed *rfidDetection =
io.feed("rfid-detection");
AdafruitIO_Feed *scanCount =
io.feed("scan-count");

//
=====

= // ESP32 RFID PINS

//
=====

= //

// MFRC522 ESP32
// -----
// SDA/SS -> GPIO 5
// SCK -> GPIO 18
// MOSI -> GPIO 23
// MISO -> GPIO 19
// RST -> GPIO 22
// 3.3V -> 3.3V
// GND -> GND

//

//
=====

= #define SS_PIN 5
#define RST_PIN 22
#define BUZZER_PIN 4
MFRC522 rfid(SS_PIN, RST_PIN);

```

```
//  
=====
```

= // INVENTORY VARIABLES

```
//  
=====
```

= int totalScans = 0;

// true = material is IN

// false = material is OUT

bool cementStatus = true;

bool steelStatus = true;

bool bricksStatus = true;

```
//  
=====
```

= // FUNCTION DECLARATION

```
//  
=====
```

=

// IMPORTANT: This fixes the

// "beepBuzzer was not declared in this scope" error.

void beepBuzzer(int times);

```
//  
=====
```

= // SETUP

```
//  
=====
```

=

void setup()

{

Serial.begin(9600);

delay(1000);

```

// =====
// BUZZER

// =====

pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(BUZZER_PIN, LOW);

// =====
// START SPI

// =====
SPI.begin(18, 19, 23, 5);

// =====
// START RFID

// =====
rfid.PCD_Init();

delay(500);

Serial.println();

Serial.println("=====");
Serial.println(" SMART INVENTORY MANAGEMENT SYSTEM");
Serial.println(" ESP32 + MFRC522 + ADAFRUIT IO");
Serial.println("=====");

// =====
// RFID TEST

// =====
byte version = rfid.PCD_ReadRegister(rfid.VersionReg);
Serial.print("MFRC522 Version: 0x");

Serial.println(version, HEX);

if (version == 0x00 || version == 0xFF)
{
Serial.println("ERROR: MFRC522 NOT DETECTED!");
Serial.println("Check RFID wiring.");
}

```

```

    }
    else
    {
        Serial.println("MFRC522 detected successfully.");
    }

    // =====
    // CONNECT TO ADAFRUIT IO
    //
    // =====

    Serial.println();
    Serial.println("Connecting to Adafruit IO...");
    io.connect();
    while (io.status() < AIO_CONNECTED)
    {
        Serial.print(".");
        delay(500);
    }
    Serial.println();
    Serial.println("Adafruit IO Connected!");

    // =====
    // INITIAL CLOUD STATUS
    // =====
    rfidDetection->save("NO TAG");
    inventoryStatus->save("READY");
    scanCount->save(totalScans);
    Serial.println();
    Serial.println("RFID Reader Ready");

```



```
Serial.println("Scan RFID Tag...");
Serial.println();
}
//
=====
= // LOOP
//
=====
=
void loop()
{
// =====
// KEEP ADAFRUIT IO CONNECTION ALIVE
// =====
io.run();
// =====
// CHECK FOR NEW RFID CARD
//
=====

if (!rfid.PICC_IsNewCardPresent())
{
return;
}
// =====
// READ RFID CARD
//
=====

if (!rfid.PICC_ReadCardSerial())
{
```

```

return;

}

Serial.println("-----");

Serial.println("RFID CARD DETECTED!");

// =====
// READ UID
// =====

String uidString = "";

for (byte i = 0; i < rfid.uid.size; i++)
{
  if (i > 0)
  {
    uidString += " ";
  }

  if (rfid.uid.uidByte[i] < 0x10)
  {
    uidString += "0";
  }

  uidString += String(rfid.uid.uidByte[i], HEX);
}

uidString.toUpperCase();

// =====
// DISPLAY UID
//
=====

Serial.print("RFID UID: ");

Serial.println(uidString);

```

```

// =====
// SEND RFID DETECTION

// =====
rfidDetection->save("DETECTED");

// =====
// SEND UID TO ADAFRUIT IO

// =====
rfidUID->save(uidString);

// =====
// INCREASE SCAN COUNT

// =====
totalScans++;

scanCount->save(totalScans);

// =====
// MATERIAL IDENTIFICATION

//
=====

String material = "UNKNOWN";

String status = "UNKNOWN";

// =====
// CEMENT RFID

//
=====

if (uidString == "23 4A 91 7C")
{
    material = "CEMENT";
    if (cementStatus == true)
    {
        status = "OUT";
    }
}

```

```
cementStatus = false;
}
else
{
status = "IN";
cementStatus = true;
}
}
// =====
// STEEL RFID
//
=====

else if (uidString == "11 22 33 44")
{
material = "STEEL";
if (steelStatus == true)
{
status = "OUT";
steelStatus = false;
}
else
{
status = "IN";
steelStatus = true;
}
}
```

```
// =====  
// BRICKS RFID  
//  
=====
```



```
else if (uidString == "55 66 77 88")  
{  
    material = "BRICKS";  
    if (bricksStatus == true)  
    {  
        status = "OUT";  
        bricksStatus = false;  
    }  
    else  
    {  
        status = "IN";  
        bricksStatus = true;  
    }  
}  
  
// =====  
// UNKNOWN RFID  
//  
=====
```



```
else  
{  
    material = "UNKNOWN";  
    status = "UNAUTHORIZED";  
}
```

```

// =====
// SERIAL OUTPUT
// =====

Serial.print("Material: ");
Serial.println(material);
Serial.print("Inventory Status: ");
Serial.println(status);

// =====
// SEND DATA TO ADAFRUIT IO
// =====
materialName->save(material);
inventoryStatus->save(status);

// =====
// BUZZER
//
=====

if (material == "UNKNOWN")
{
// Two beeps for unauthorized RFID
beepBuzzer(2);
}
else
{
// One beep for valid RFID
beepBuzzer(1);
}

Serial.println("Data sent to Adafruit IO");
Serial.println("-----");

```

```

// =====
// STOP RFID COMMUNICATION
//
=====

rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();
delay(1000);

// =====
// RESET CLOUD DETECTION STATUS
//
=====

rfidDetection->save("NO TAG");
}

//
=====

= // BUZZER FUNCTION

//
=====

=

void beepBuzzer(int times)
{
for (int i = 0; i < times; i++)
{
digitalWrite(BUZZER_PIN, HIGH);
delay(200);
digitalWrite(BUZZER_PIN, LOW);
delay(200);
}
}

```

```
}

RFID NUMBER //
=====

= // SMART INVENTORY MANAGEMENT SYSTEM

// ESP32 + MFRC522 RFID + BUZZER + ADAFRUIT IO

//
=====

=

#include <WiFi.h>

#include <SPI.h>

#include <MFRC522.h>

#include "AdafruitIO_WiFi.h"

//
=====

= // WIFI DETAILS

//
=====

=

#define WIFI_SSID "OnePlus Nord CE5 7C5C"

#define WIFI_PASS "qkts5737"

//
=====

= // ADAFRUIT IO DETAILS

//
=====

=

#define IO_USERNAME "Hair_13"

#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
```



```

//
=====
= // ADAFRUIT IO CONNECTION
//
=====
= AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID,
WIFI_PASS);
//
=====
= // ADAFRUIT IO FEEDS
//
=====
=
AdafruitIO_Feed *rfidUID =
io.feed("rfid-uid");
AdafruitIO_Feed *materialName =
io.feed("material-name");
AdafruitIO_Feed *inventoryStatus =
io.feed("inventory-status");
AdafruitIO_Feed *rfidDetection =
io.feed("rfid-detection");
AdafruitIO_Feed *scanCount =
io.feed("scan-count");
//
=====
= // ESP32 RFID PINS
//
=====
= //
// MFRC522 ESP32

```

```

// -----
// SDA/SS -> GPIO 5
// SCK -> GPIO 18
// MOSI -> GPIO 23
// MISO -> GPIO 19
// RST -> GPIO 22
// 3.3V -> 3.3V
// GND -> GND
//
//
=====
=
#define SS_PIN 5
#define RST_PIN 22
#define BUZZER_PIN 4
MFRC522 rfid(SS_PIN, RST_PIN);
//
=====
= // INVENTORY VARIABLES
//
=====
= int totalScans = 0;
// true = material is IN
// false = material is OUT
bool cementStatus = true;
bool steelStatus = true;
bool bricksStatus = true;

```

```
//
=====
= // FUNCTION DECLARATION
//
=====
=
// IMPORTANT: This fixes the
// "beepBuzzer was not declared in this scope" error.
void beepBuzzer(int times);
//
=====
=
// SETUP
//
=====
=
void setup()
{
  Serial.begin(9600);
  delay(1000);
  // =====
  // BUZZER
  //
  =====

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  // =====
  // START SPI
  // =====
  SPI.begin(18, 19, 23, 5);
```

```
// =====  
// START RFID  
  
// =====  
rfid.PCD_Init();  
  
delay(500);  
  
Serial.println();  
  
Serial.println("=====");  
Serial.println(" SMART INVENTORY MANAGEMENT SYSTEM");  
Serial.println(" ESP32 + MFRC522 + ADAFRUIT IO");  
  
Serial.println("=====");  
  
// =====  
// RFID TEST  
  
// =====  
  
byte version = rfid.PCD_ReadRegister(rfid.VersionReg);  
  
Serial.print("MFRC522 Version: 0x");  
  
Serial.println(version, HEX);  
  
if (version == 0x00 || version == 0xFF)  
{  
  
Serial.println("ERROR: MFRC522 NOT DETECTED!");  
Serial.println("Check RFID wiring.");  
  
}  
  
else  
  
{  
  
Serial.println("MFRC522 detected successfully.");  
  
}  
  
// =====  
// CONNECT TO ADAFRUIT IO
```

```

//
=====

Serial.println();
Serial.println("Connecting to Adafruit IO...");
io.connect();
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}
Serial.println();
Serial.println("Adafruit IO Connected!");

// =====
// INITIAL CLOUD STATUS

// =====
rfidDetection->save("NO TAG");
inventoryStatus->save("READY");
scanCount->save(totalScans);
Serial.println();
Serial.println("RFID Reader Ready");
Serial.println("Scan RFID Tag...");
Serial.println();
}

//
=====
= // LOOP

```

```

//
=====

=

void loop()
{
// =====
// KEEP ADAFRUIT IO CONNECTION ALIVE

// =====
io.run();

// =====
// CHECK FOR NEW RFID CARD

//
=====

if (!rfid.PICC_IsNewCardPresent())
{
return;
}

// =====
// READ RFID CARD

//
=====

if (!rfid.PICC_ReadCardSerial())
{
return;
}

Serial.println("-----");
Serial.println("RFID CARD DETECTED!");

// =====
// READ UID

```

```

// =====
String uidString = "";
for (byte i = 0; i < rfid.uid.size; i++)
{
  if (i > 0)
  {
    uidString += " ";
  }
  if (rfid.uid.uidByte[i] < 0x10)
  {
    uidString += "0";
  }
  uidString += String(rfid.uid.uidByte[i], HEX);
}
uidString.toUpperCase();

// =====
// DISPLAY UID

//
=====

Serial.print("RFID UID: ");
Serial.println(uidString);

// =====
// SEND RFID DETECTION

// =====
rfidDetection->save("DETECTED");

// =====
// SEND UID TO ADAFRUIT IO

```

```

// =====
rfidUID->save(uidString);

// =====
// INCREASE SCAN COUNT

// =====
totalScans++;

scanCount->save(totalScans);

// =====
// MATERIAL IDENTIFICATION

//
=====

String material = "UNKNOWN";
String status = "UNKNOWN";

// =====
// CEMENT RFID

//
=====

if (uidString == "23 4A 91 7C")
{
    material = "CEMENT";
    if (cementStatus == true)
    {
        status = "OUT";
        cementStatus = false;
    }
    else
    {
        status = "IN";
    }
}

```



```
cementStatus = true;

}

}

// =====

// STEEL RFID

//
=====

else if (uidString == "11 22 33 44")
{
material = "STEEL";
if (steelStatus == true)
{
status = "OUT";
steelStatus = false;
}
else
{
status = "IN";
steelStatus = true;
}
}

// =====

// BRICKS RFID

//
=====

else if (uidString == "55 66 77 88")
{
```

```
material = "BRICKS";
if (bricksStatus == true)
{
status = "OUT";
bricksStatus = false;
}
else
{
status = "IN";
bricksStatus = true;
}
}

// =====

// UNKNOWN RFID

//
=====

else
{
material = "UNKNOWN";
status = "UNAUTHORIZED";
}

// =====
// SERIAL OUTPUT

//
=====

Serial.print("Material: ");
Serial.println(material);
```

```

Serial.print("Inventory Status: ");

Serial.println(status);

// =====
// SEND DATA TO ADAFRUIT IO
// =====
materialName->save(material);
inventoryStatus->save(status);

// =====
// BUZZER

//
=====

if (material == "UNKNOWN")
{
// Two beeps for unauthorized RFID
beepBuzzer(2);
}
else
{
// One beep for valid RFID
beepBuzzer(1);
}

Serial.println("Data sent to Adafruit IO");
Serial.println("-----");

// =====
// STOP RFID COMMUNICATION
//
=====

rfid.PICC_HaltA();

```

```

rfid.PCD_StopCrypto1();
delay(1000);

// =====
// RESET CLOUD DETECTION STATUS
//
=====

rfidDetection->save("NO TAG");
}

//
=====

= // BUZZER FUNCTION

//
=====

=

void beepBuzzer(int times)
{
for (int i = 0; i < times; i++)
{
digitalWrite(BUZZER_PIN, HIGH);
delay(200);
digitalWrite(BUZZER_PIN, LOW);
delay(200);
}
}

```

ADAFRUIT IO CLOUD FEEDS

The following feeds can be created in the cloud dashboard:

1. material-id

Type: Text

Purpose: Displays the RFID/material identification number.

2. material-name

Type: Text

Purpose: Displays the name of the construction material.

3. material-quantity

Type: Number

Purpose: Displays the current quantity or weight of the material.

4. movement-type

Type: Text

Purpose: Displays STOCK-IN, STOCK-OUT or TRANSFER.

5. stock-status

Type: Text

Purpose: Displays NORMAL, LOW STOCK or OUT OF STOCK.

6. storage-location

Type: Text

Purpose: Displays the location where the material is stored.

7. inventory-alert

Type: Number / Text

Purpose: Displays the inventory warning or alert condition.

SYSTEM WORKING

The system begins by initializing the ESP32, RFID reader, HX711 module, OLED display and alert system.

When a material is brought into the inventory area, its RFID tag is scanned by the RFID reader. The ESP32 identifies the material according to the RFID UID.

The material is then placed on the weighing platform. The load cell measures its weight and sends the measurement to the HX711 module.

The HX711 sends the digital measurement to the ESP32.

The ESP32 calculates the quantity and updates the inventory information.

During stock-in, the new quantity is added to the available inventory.

During stock-out, the consumed quantity is deducted from the available inventory.

The ESP32 then determines the current inventory status.

If sufficient material is available, the status is shown as NORMAL.

If the quantity falls below the predefined minimum level, the status becomes LOW STOCK.

If the material quantity reaches zero, the status becomes OUT OF STOCK.

The inventory information is transmitted through Wi-Fi to the cloud platform. The dashboard provides real-time information about the available materials and their movement.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	ESP32 Dev Board	1	Main controller and Wi-Fi communication
2	MFRC522 RFID Reader	1	Material identification
3	RFID Tags / Cards	As required	Identification of materials
4	Load Cell	1	Measures material quantity
5	HX711 Module	1	Load-cell signal processing
6	OLED Display	1	Local inventory display
7	Buzzer	1	Low-stock warning
8	LED	1 set	Status indication
9	Breadboard	1	Circuit assembly
10	Jumper Wires	1 set	Circuit connections
11	Power Supply	1	Power source
12	Storage / Weighing Platform	1	Supports the material during measurement

PROTOTYPE IMPLEMENTATION

The prototype is designed around an ESP32 controller with RFID identification and a load-cell based weighing platform.

The RFID reader is installed near the material storage or inventory area. Each material or material container is assigned an RFID tag.

The load cell is positioned below the material platform to measure the material quantity.

When the RFID tag is detected, the ESP32 identifies the material. The load cell measures its current quantity.

The OLED display provides local information such as:

Material: Cement

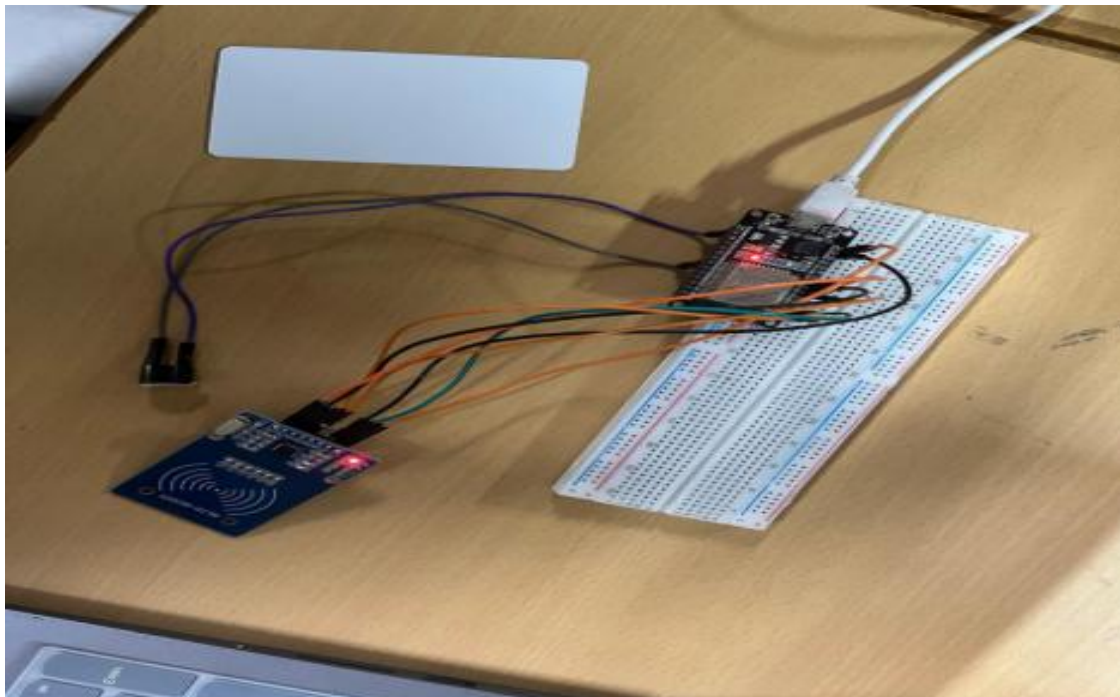
Quantity: 18.5 kg

Status: NORMAL

The ESP32 communicates with the cloud platform through Wi-Fi.

The cloud dashboard displays the inventory data and allows the user to monitor material availability remotely.

The prototype can be expanded by adding multiple RFID tags and separate storage locations for different construction materials.



RESULTS & PERFORMANCE ANALYSIS

The developed Smart Inventory Management System provides real-time monitoring of construction-material inventory.

The system can identify the material using RFID and measure the quantity using the load cell.

The inventory condition can be classified according to the available quantity.

Parameter	Normal	Warning	Critical
Material Quantity	Above minimum level	At or below minimum level	Critical reserve / zero
Stock Status	NORMAL	LOW STOCK	OUT OF STOCK
Inventory Alert	OFF	ON	ON

The system performance can be evaluated based on:

- RFID identification accuracy.
- Load-cell measurement stability.
- Inventory calculation accuracy.
- Correct stock-in detection.
- Correct stock-out detection.
- Cloud communication reliability.
- Dashboard update performance.
- Low-stock alert operation.

PERFORMANCE

- Provides real-time construction material inventory monitoring.
- Uses RFID for material identification.
- Uses a load cell for quantity measurement.
- Tracks stock-in and stock-out movement.
- Provides cloud-based remote monitoring.
- Displays current material quantity.
- Provides low-stock alerts.
- Stores historical inventory information.

- Helps reduce manual inventory recording.
- Helps prevent unexpected material shortages.
- Helps identify material consumption patterns.
- Improves resource utilization and site management efficiency.
- Provides a low-cost prototype suitable for educational and small-scale construction applications.

